Manipulación de datos (DML)

Agregar, leer, cambiar y borrar filas con `INSERT`, `SELECT`, `UPDATE`, `DELETE` y `TRUNCATE`

Semana 07 - Sesión 14

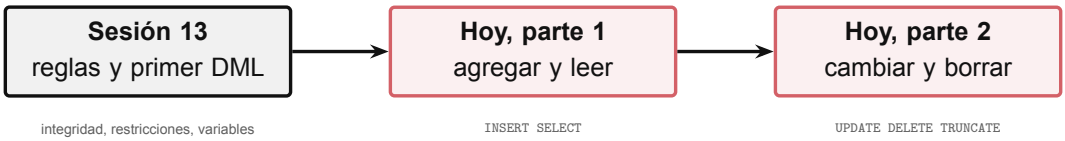
Rel Guzman



Logro de la sesión

Al finalizar, podrás **agregar** filas con `INSERT`, **leerlas** con `SELECT`, **cambiarlas** con `UPDATE`, **borrarlas** con `DELETE` y **vaciar** una tabla con `TRUNCATE`, sabiendo cuándo conviene usar cada una.

Dónde estamos



En palabras simples

La sesión pasada vimos DML de pasada, como quien prueba el control remoto. Hoy aprendemos **cada botón con calma**: qué hace, cómo se escribe y en qué se equivoca uno al empezar.

DML

DDL construye, DML llena

DDL: el envase

Crea y cambia la **forma** de las tablas: qué columnas tienen, de qué tipo, con qué reglas.

```
CREATE TABLE …
ALTER TABLE …
DROP TABLE …
```

DML: el contenido

Trabaja con las **filas**: las agrega, las lee, las cambia y las borra. La tabla sigue igual.

```
INSERT    SELECT
UPDATE    DELETE
TRUNCATE
```

En palabras simples

Un armario es el **DDL**: sus repisas y cajones. La ropa que metes, sacas o cambias de lugar es el **DML**. Puedes ordenar la ropa mil veces sin tocar el armario.

DML = *Data Manipulation Language*, lenguaje de manipulación de datos.

Cinco palabras para cinco acciones

`INSERT`	`SELECT`	`UPDATE`	`DELETE`	`TRUNCATE`
agrega filas nuevas	**lee** sin cambiar nada	**cambia** datos que ya están	**borra** las filas que elijas	**vacía** toda la tabla
llega un producto	ver el catálogo	sube un precio	se descontinúa uno	empezar de cero

Cuatro de ellas **cambian** los datos. Solo `SELECT` se limita a **mirar**: por eso es la más segura para practicar.

Cómo se ve en la realidad

Cada vez que compras en línea pasan estas cosas: tu pedido se **inserta**, el catálogo se **consulta**, el stock se **actualiza** y tu carrito, al pagar, se **borra**.

La tabla de ejemplo: una bodega

```sql
CREATE TABLE producto (
   id        INT PRIMARY KEY AUTO_INCREMENT,
   nombre    VARCHAR(60) NOT NULL,
   categoria VARCHAR(30),
   precio    DECIMAL(6,2) NOT NULL,
   stock     INT DEFAULT 0
);
```

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50

Cada ejemplo parte de **estas cuatro filas**, y al lado verás **cómo queda la tabla** después de la instrucción. Lo que cambia se marca en amarillo.

Ojo con esto

Los ejemplos están en MySQL. `INSERT`, `SELECT`, `UPDATE` y `DELETE` **se escriben igual** en SQL Server y SQLite; las pocas diferencias se señalan en su diapositiva.

INSERT

INSERT: tres preguntas, tres partes

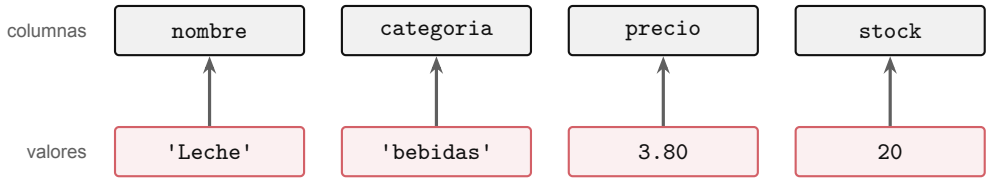
```
INSERT INTO producto
(nombre, categoria, precio, stock)
VALUES ('Leche', 'bebidas', 3.80, 20);
```

- ¿En **qué tabla**?
- ¿Qué **columnas** voy a llenar?
- ¿Con **qué datos**?

El `id` no aparece: lo pone el **autoincremento**. La fila nueva se suma al final.

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20

Cada valor va a su columna, en orden



- El **primer** valor va a la **primera** columna, el segundo a la segunda, y así.
- Debe haber **tantos valores como columnas**: ni uno más, ni uno menos.
- El texto va entre **comillas simples** `'así'`. Las fechas también: `'2026-09-26'`.

En palabras simples

Es como llenar un formulario en papel: cada dato va en **su casillero**. Si escribes tu edad donde iba tu nombre, el formulario queda mal aunque «tenga todos los datos».

Práctica 1 Encuentra el error

Cada `INSERT` tiene **un** error. Escribe cuál es.

Instrucción	¿Qué está mal?
`INSERT INTO producto (nombre, precio) VALUES ('Pan', 0.30, 100);`	
`INSERT INTO producto (nombre, precio) VALUES (Pan, 0.30);`	
`INSERT INTO producto (nombre, precio) VALUES (0.30, 'Pan');`	
`INSERT INTO producto (categoria, precio) VALUES ('snacks', 2.00);`	

Pista: revisa la cantidad de valores, las comillas, el orden y las reglas de la tabla (¿qué columna era `NOT NULL`?).

SELECT

SELECT: ¿qué columnas y de qué tabla?

```sql
-- todas las columnas
SELECT * FROM producto;
```

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50

```sql
-- solo las que me interesan
SELECT nombre, precio FROM producto;
```

nombre	precio
Arroz	4.50
Gaseosa	3.00
Agua	1.50
Galletas	1.20

- `*` se lee «**todas las columnas**». Útil para echar un vistazo rápido.
- Con nombres, el resultado trae **solo esas** columnas y **en ese orden**.
- `SELECT` **nunca** cambia la tabla: el resultado es una copia para mirar.

WHERE: quedarse solo con algunas filas

```sql
SELECT nombre, precio FROM producto WHERE precio < 2;
```

nombre	precio
Agua	1.50
Galletas	1.20

```sql
SELECT nombre FROM producto WHERE categoria = 'bebidas';
```

nombre
Gaseosa
Agua

=	<>	>	<	>=	<=
igual	distinto	mayor	menor	mayor o igual	menor o igual

En palabras simples

WHERE es un **colador**: la tabla pasa fila por fila y solo quedan las que **cumplen** la condición. Las demás siguen en la tabla; simplemente no se muestran.

`ORDER BY`: ordenar el resultado

```sql
SELECT nombre, precio FROM producto
ORDER BY precio;
```

nombre	precio
Galletas	1.20
Agua	1.50
Gaseosa	3.00
Arroz	4.50

De menor a mayor (`ASC`, el normal).

```sql
SELECT nombre, precio FROM producto
ORDER BY precio DESC;
```

nombre	precio
Arroz	4.50
Gaseosa	3.00
Agua	1.50
Galletas	1.20

De mayor a menor (`DESC`, descendente).

Sin `ORDER BY`, el orden no está garantizado. Suele salir en el orden en que se guardó, pero el gestor no lo promete. Si el orden importa, **pídelo**.

Con texto ordena alfabéticamente: `ORDER BY nombre` da Agua, Arroz, Galletas, Gaseosa.

El orden de las piezas de un `SELECT`

```
SELECT   nombre, precio
FROM     producto
WHERE    stock > 0
ORDER BY precio DESC;
```

- ¿Qué **columnas** quiero ver?
- ¿De qué **tabla**?
- ¿Qué **filas** me quedo?
- ¿En qué **orden**?

Solo `SELECT` y `FROM` son obligatorias. Las demás son **opcionales**, pero si aparecen, van **en este orden**. Escribir `ORDER BY` antes de `WHERE` da error.

¿Qué filas salen?

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50

Escribe los **nombres** que devuelve cada consulta. Si no sale ninguno, escribe «ninguno».

Consulta	Resultado
`SELECT nombre FROM producto WHERE stock >= 30;`	_____
`SELECT nombre FROM producto WHERE categoria <> 'bebidas';`	_____
`SELECT nombre FROM producto WHERE stock = 0;`	_____
`SELECT nombre FROM producto WHERE precio > 5;`	_____

Práctica 3 Escribe la consulta

Sobre la misma tabla `producto`, escribe el `SELECT` para cada pedido del bodeguero:

1. «Muéstrame **todo** lo que tengo de la categoría `abarrotes`».
2. «Dame **nombre y stock** de lo que tiene menos de 15 unidades».
3. «Lista los productos **del más barato al más caro**».
4. «¿Qué productos cuestan **más de S/ 2**?».

```
SELECT _____ FROM producto
…
```

Pista: primero responde **qué columnas**, luego **qué filas**, luego **en qué orden**.

UPDATE

UPDATE: cambiar lo que ya está

```
UPDATE producto
SET    precio = 3.50
WHERE  id = 2;
```

- ¿En **qué tabla**?
- ¿**Qué cambio**?
- ¿A **qué filas**?

La gaseosa subió de precio. Solo cambia **esa celda**: el resto de la fila y las demás filas quedan igual.

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.50	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50

En palabras simples

Es como corregir con **corrector líquido**: no arrancas la hoja, solo tapas el dato viejo y escribes el nuevo encima.

El olvido más caro: UPDATE sin WHERE

```
UPDATE producto SET precio = 3.50;
-- ¡se olvidó el WHERE!
```

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	3.50	30
2	Gaseosa	bebidas	3.50	12
3	Agua	bebidas	3.50	0
4	Galletas	snacks	3.50	50

Ojo con esto

Sin `WHERE`, **todas las filas** reciben el cambio. Toda la bodega pasa a costar S/ 3.50 y los precios anteriores **se perdieron**: no hay «deshacer».

Buena costumbre: primero mira, luego cambia

Prueba el `WHERE` con un `SELECT`. Si salen justo las filas que querías, recién haz el `UPDATE`.

```
SELECT * FROM producto WHERE id = 2;      -- 1. ¿son estas?
UPDATE producto SET precio = 3.50 WHERE id = 2;  -- 2. ahora sí
```

Práctica 4 Escribe el `UPDATE`

Partiendo de la tabla original de `producto`, escribe la instrucción para cada situación:

Situación	**Tu `UPDATE`**
Las galletas ahora cuestan S/ 1.30.	
Ahora hay 24 botellas de agua.	
Todas las bebidas cuestan S/ 2.00.	
El arroz pasa a la categoría `granos`.	

Pista: ¿qué **columna** cambia, a qué **valor**, y en qué **filas**?

DELETE

DELETE: borrar filas

```
DELETE FROM producto
WHERE       stock = 0;
```

◀ ¿De **qué tabla**?

◀ ¿**Cuáles** filas se van?

- Se borra la **fila entera**, con todas sus columnas.
- No se escriben columnas: `DELETE nombre FROM…` es un error.

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
~~3~~	~~Agua~~	~~bebidas~~	~~1.50~~	~~0~~
4	Galletas	snacks	1.20	50

En palabras simples

¿Y si solo quiero **vaciar un dato**, no la fila? Eso no es `DELETE`, es un `UPDATE`: `UPDATE producto SET categoria = NULL WHERE id = 3;`

Antes de borrar, tres cuidados

1. No olvides el WHERE

`DELETE FROM producto;` borra **todas** las filas. La tabla sigue existiendo, pero vacía.

2. No hay papelera

Lo borrado **no se recupera** con un clic. Mira antes con un `SELECT` y el mismo `WHERE`.

3. Las claves foráneas

Si una `venta` apunta al producto, el gestor **no deja borrarlo** (salvo `ON DELETE CASCADE`).

```
SELECT * FROM producto WHERE stock = 0;     -- 1. ¿cuáles se irían?
DELETE FROM producto WHERE stock = 0;       -- 2. ahora sí
```

Práctica 5 ¿Cuántas filas se borran?

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50

Cada instrucción parte de la tabla **completa**. ¿Cuántas filas borra y cuáles?

Instrucción	¿Cuántas?	¿Cuáles?
`DELETE FROM producto WHERE id = 7;`	_____	_____
`DELETE FROM producto WHERE categoria = 'bebidas';`	_____	_____
`DELETE FROM producto WHERE precio > 1;`	_____	_____
`DELETE FROM producto;`	_____	_____

Pista: un `WHERE` que no encuentra filas **no da error**: simplemente borra cero.

TRUNCATE

TRUNCATE: vaciar la tabla de golpe

```sql
TRUNCATE TABLE producto;
```

- Borra **todas** las filas, siempre. No acepta `WHERE`.
- La tabla **sigue existiendo**, con sus columnas y reglas.
- El autoincremento **vuelve a empezar** en 1.
- Es **más rápido** que `DELETE` en tablas grandes.

producto

id	nombre	categoria	precio	stock
		(vacía)		

En palabras simples

`DELETE` es borrar renglones de un cuaderno **con borrador**, uno por uno, eligiendo cuáles. `TRUNCATE` es **arrancar todas las hojas escritas**: el cuaderno queda como nuevo y vuelves a la página 1.

Funciona igual en MySQL y SQL Server. SQLite no lo tiene: ahí se usa `DELETE FROM producto;`.

`DELETE`, `TRUNCATE` y `DROP`: no son lo mismo

	DELETE	TRUNCATE	DROP TABLE
¿Qué borra?	las filas **que elijas**	**todas** las filas	la tabla **entera**
¿Acepta `WHERE`?	sí	no	no
¿La tabla sigue?	sí	sí	**no**
¿El `id` vuelve a 1?	no, sigue contando	**sí**	ya no hay tabla
¿Con tablas grandes?	lento	rápido	rápido



Elige la instrucción adecuada para cada caso y escríbela:

Caso	¿Cuál?	Instrucción
La bodega dejó de vender galletas.		
Terminé de practicar y quiero la tabla vacía, con el `id` desde 1.		
La tabla `producto_viejo` ya no se usa para nada.		
Quitar todo lo que tenga stock 0.		

Pista: pregúntate: ¿borro **algunas** filas, **todas** las filas o la **tabla misma**?

Preguntas



Todo junto

Resumen: las cinco instrucciones

Instrucción	Qué hace	Forma básica	¿Cambia datos?
`INSERT`	agrega filas	`INSERT INTO t (a, b) VALUES (1, 2);`	sí
`SELECT`	lee filas	`SELECT a, b FROM t WHERE …;`	**no**
`UPDATE`	cambia valores	`UPDATE t SET a = 1 WHERE …;`	sí
`DELETE`	borra filas elegidas	`DELETE FROM t WHERE …;`	sí
`TRUNCATE`	vacía la tabla	`TRUNCATE TABLE t;`	sí

`WHERE` **es el mismo** en `SELECT`, `UPDATE` y `DELETE`: aprendes a filtrar una vez y sirve para las tres.

La regla de oro: antes de un `UPDATE` o `DELETE`, prueba su `WHERE` con un `SELECT`.

Práctica 7 ¿Cómo queda la tabla?

Parte de la tabla original de `producto`. Ejecuta en orden y dibuja **cómo queda** al final:

```sql
-- 1
INSERT INTO producto (nombre, categoria, precio, stock)
VALUES ('Pan', 'abarrotes', 0.30, 100);

-- 2
UPDATE producto SET stock = 22 WHERE id = 2;

-- 3
DELETE FROM producto WHERE precio < 1.50;

-- 4
SELECT nombre, stock FROM producto
WHERE stock > 20 ORDER BY stock DESC;
```

id	nombre	precio	stock

¿Qué muestra el `SELECT` del paso 4, y en qué orden? _____

Ojo con el paso 3: ¿el agua, que cuesta justo 1.50, se borra o se queda?

Sigue practicando: `aprendesql.dev`



- Curso de SQL **gratuito** y **en español**.
- Cada lección trae un **editor en el navegador**: escribes la consulta, la ejecutas y ves el resultado al instante.
- Para repasar lo de hoy: los módulos **Fundamentos** (`SELECT`, `WHERE`, `ORDER BY`) y **Manipulación** (`INSERT`, `UPDATE`, `DELETE`).

`aprendesql.dev`

SQL se aprende **escribiendo**: diez minutos diarios valen más que una noche antes del examen.

Cierre: completa en voz alta

- DDL construye la tabla; DML _____
- En un `INSERT`, cada valor va _____
- Para leer solo algunas filas uso _____, y para ordenarlas _____
- Si olvido el `WHERE` en un `UPDATE` o `DELETE`…_____
- `TRUNCATE` se diferencia de `DELETE` en que _____
- Antes de cambiar o borrar, conviene probar con un _____

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.



UTP Universidad Tecnológica del Perú